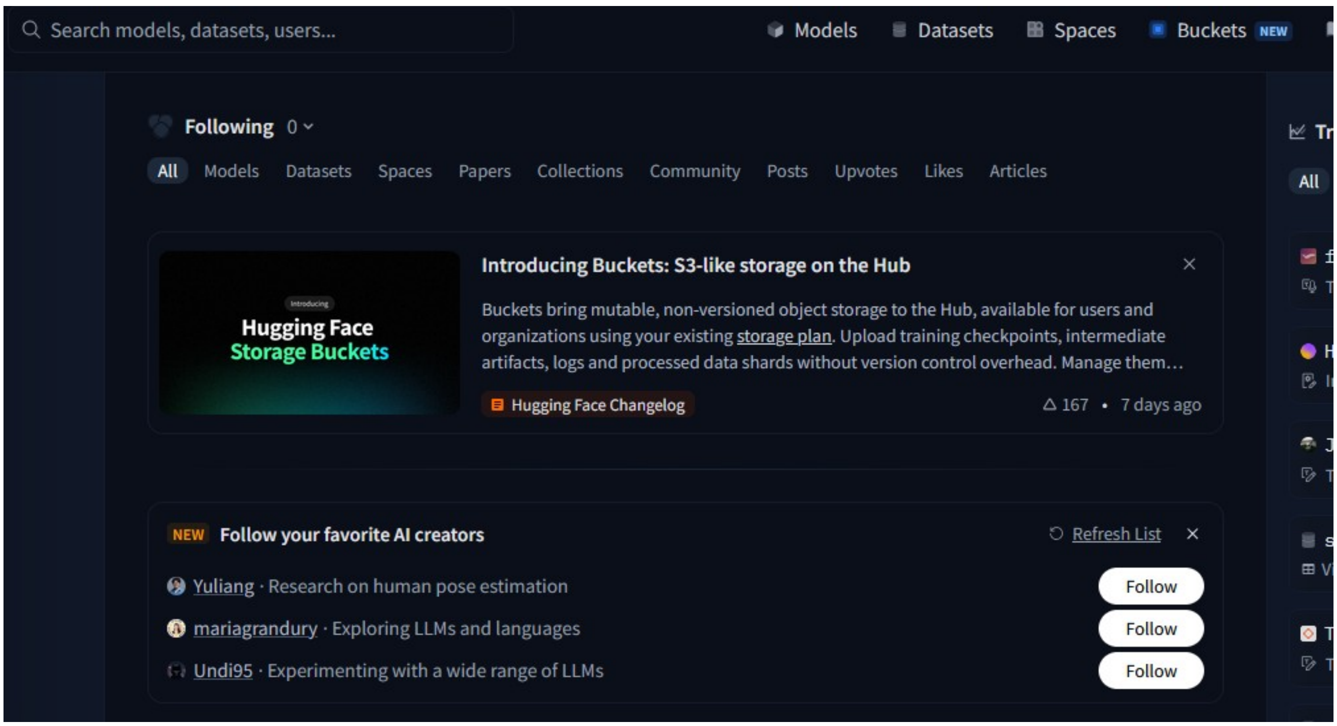
Hugging Face demo

https://huggingface.co/

We will use HuggingFace with Python Jupyter notebook

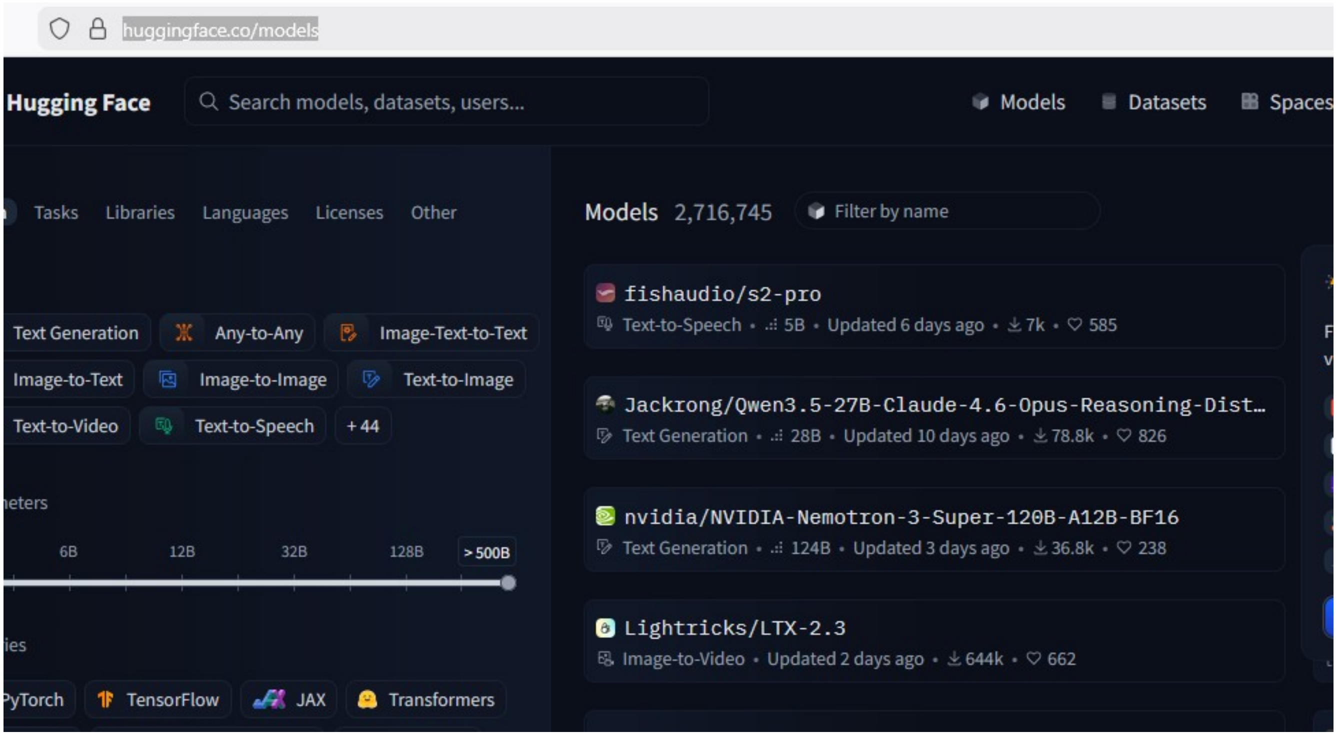
In the development world, one of the things we use it GitHub, we use GitHub to create a repository where in we can share the code with the team-member. In the same way, if you want to talk about Machine learning models, dataset etc, we will do it on HuggingFace.

Hugging Face is a prominent AI company and collaborative platform that serves as a central hub for the open-source machine learning and data science community. It is often described as the "GitHub of AI" because it allows users to share, discover, and collaborate on AI models, datasets, and applications.



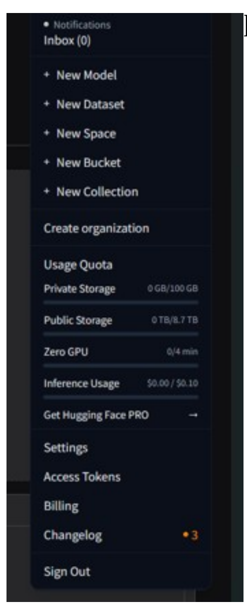
Ollama is for running open-source models locally. In the same way, if you want to run more models, maybe locally or colab, HuggingFace has different options

https://huggingface.co/models

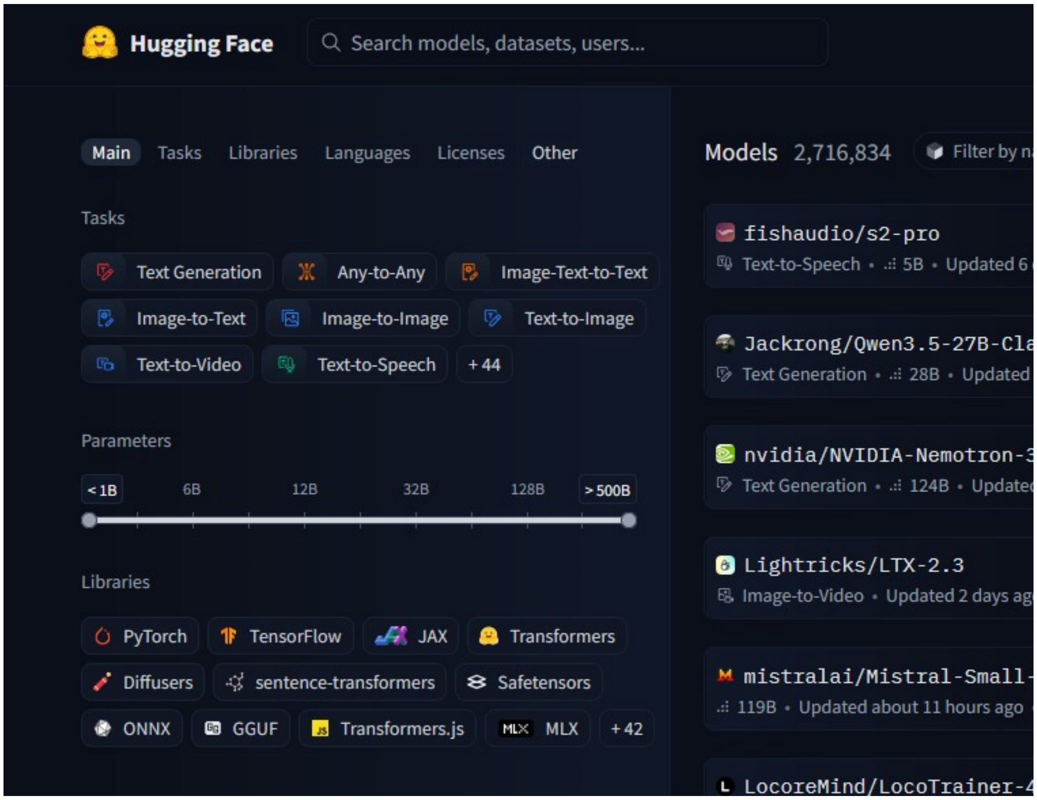


We talk about different models say from Gemini, OpenAI, Anthropic etc, say Anthropic only has text-generation models (text-to-text only, no image). But what if you are doing a task, which needs only image processing or only text-to-speech, in that case, instead of using entire large model (you could be wasting a lot of resources trying to run that model), we could use a small model and the only job of that model is to do that task for us. For OpenAI, we use a term called as 'Model-As-A-Service' that means we run the models on their servers and you pay only for the use. And they charge us based on tokens. The other option is, we can run models on local machines or push our model onto some cloud servers and run there. On HuggingFace, we can host our models on Spaces and run them.
https://huggingface.co/spaces

For example, Omni for text-to-video: https://huggingface.co/spaces/FrameAI4687/Omni-Video-Factory
It will run on HuggingFace server. HuggingFace gives you some quota of GPU time. It also offers private storage of 100GB



Inference usage is when you talk to the models



Also you can filter models based on what type of task you are going to run, based on number of parameters.

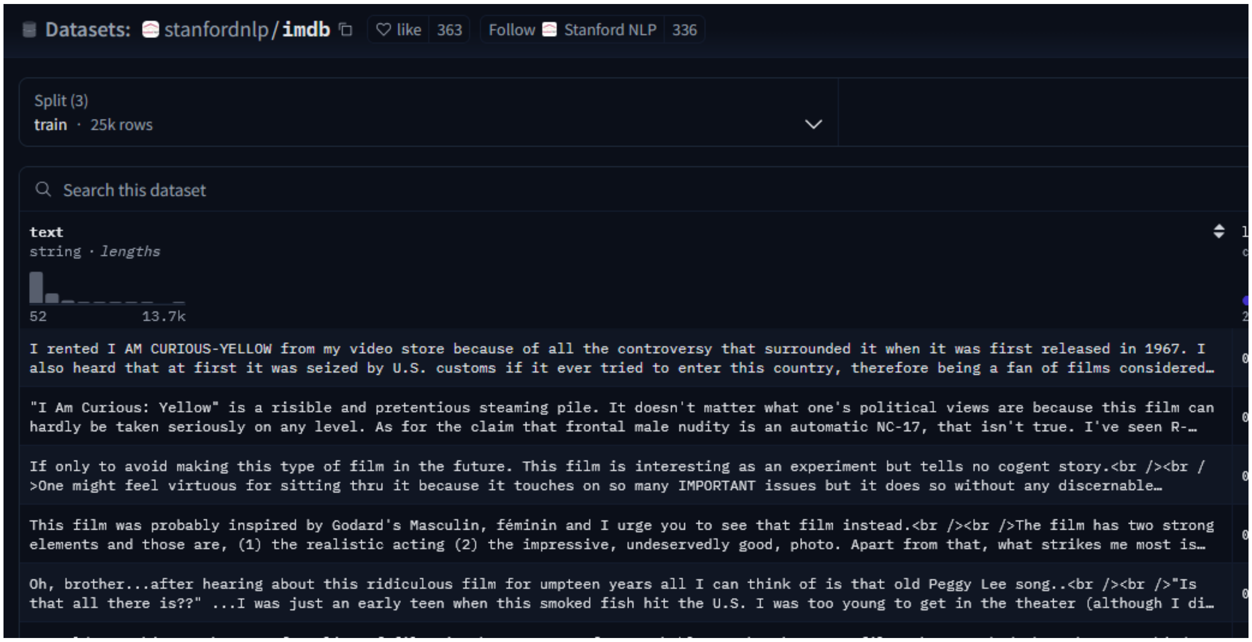
Llama3 model is free as well, but we got to fill out the form to get access https://huggingface.co/meta-llama/Llama-3.2-1B
https://huggingface.co/settings/gated-repos to check our permissions (Gated repos)

We got to fine-tune the models, either we can tune with our own data or some data available https://huggingface.co/datasets

Stanford IMDB dataset is the official one

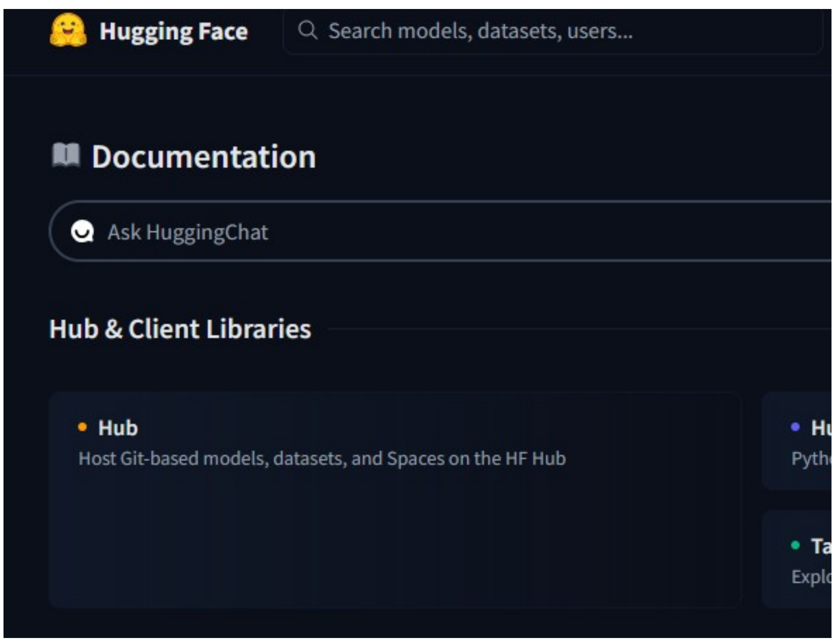
https://huggingface.co/datasets/stanfordnlp/imdb/viewer

In Data studio, we can see all the data



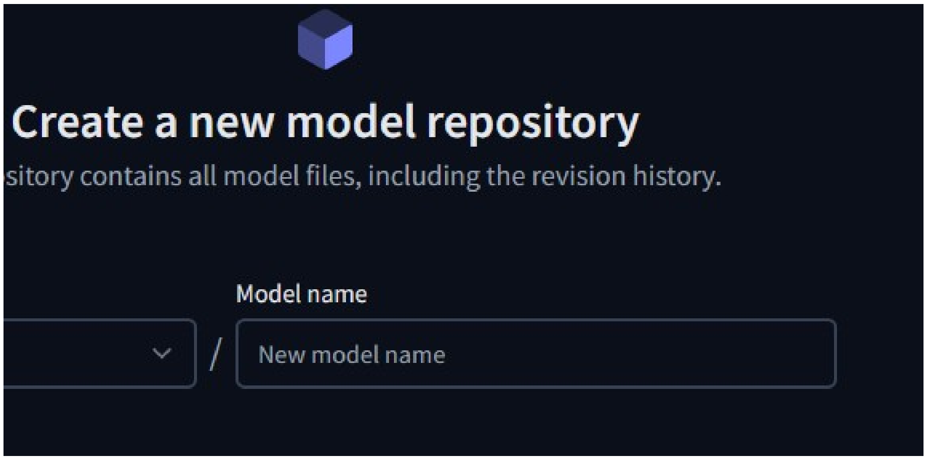
The dataset also has labels like 0 neg, 1 pos

Docs are very good. If you want to use HuggingFace, we have to connect our application to HuggingFace. To connect, we got to use HuggingFace hub.
https://huggingface.co/docs



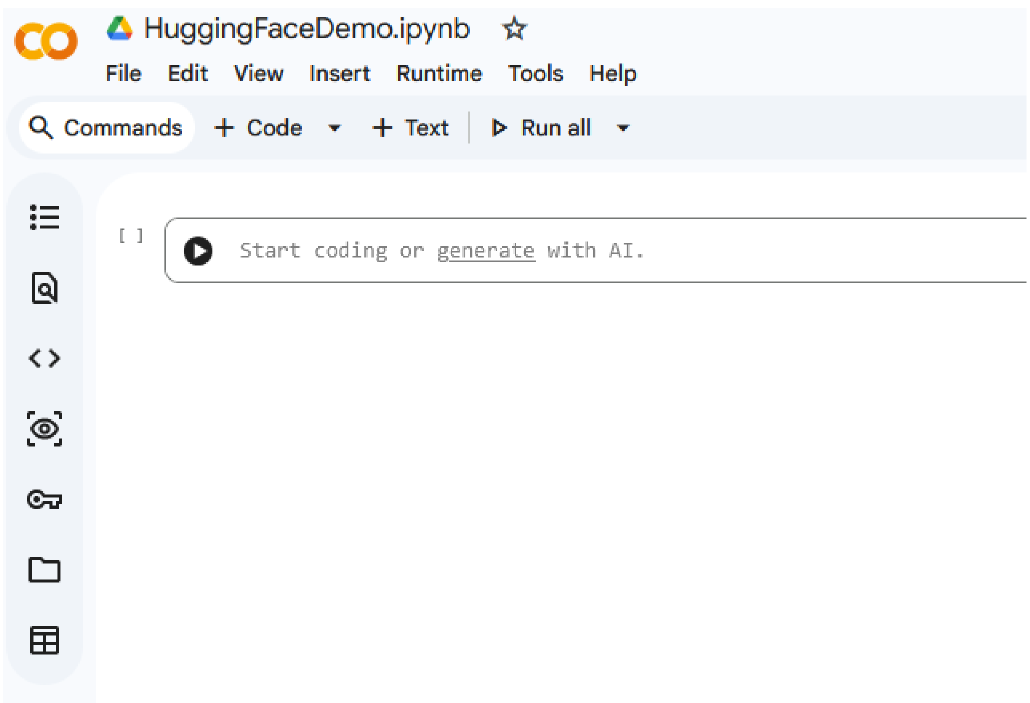
https://huggingface.co/docs/hub/index
if you want to create a web app for your ML demos, we can use Gradio

if you want to deploy your own model, we can do it here: https://huggingface.co/new

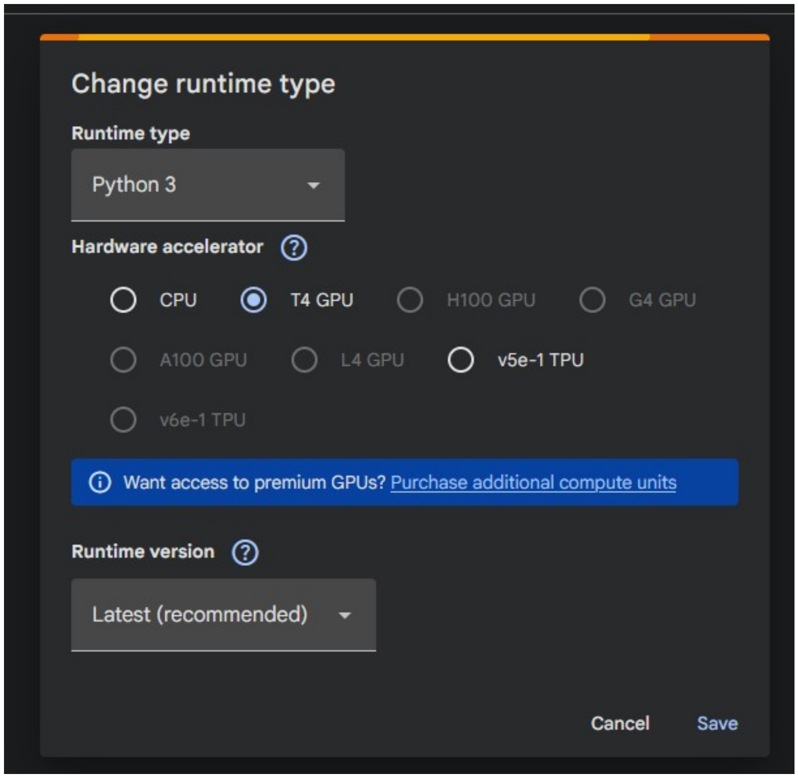


Similar to GitHub repo, we first create an ML model repo then push our model for testing or sharing like Git. We could also download a model, fine-tune and push it there

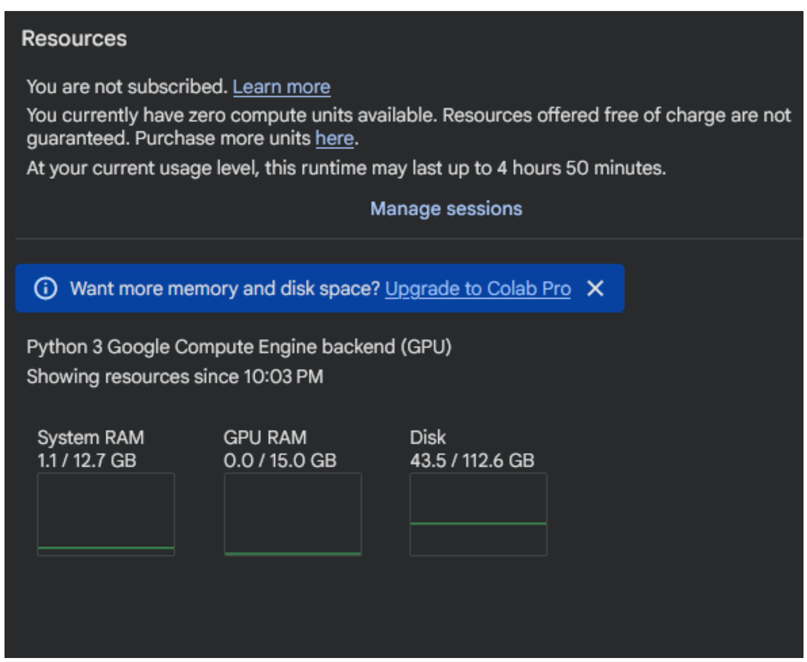
Lets use Google collab for running models



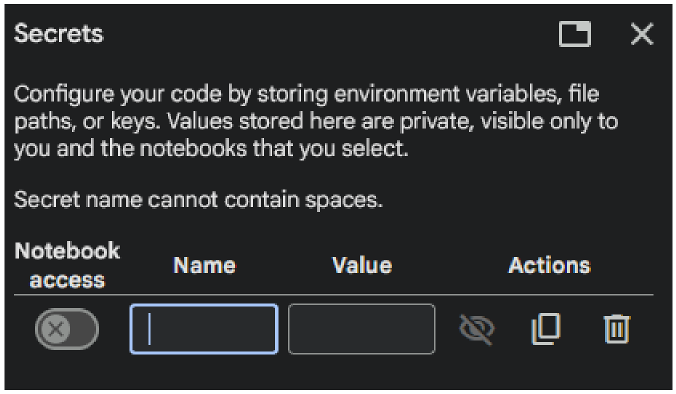
Click Connect



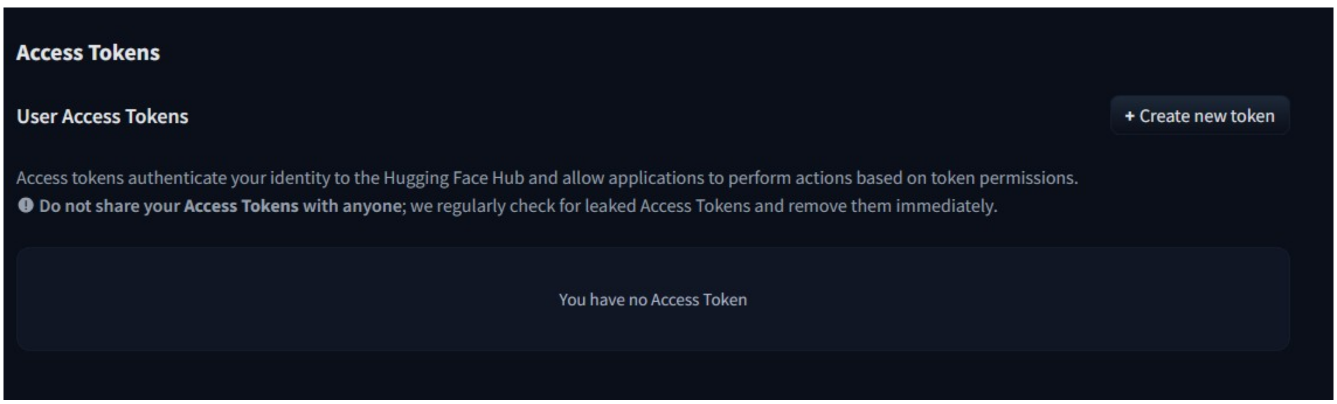
Click View resources



First we need to connect with Hugging Face
if Colab says it doesn't have access to Hugging Face model, you got to look into keys



To get access tokens (one for read and write),
Go to Hugging Face => Access tokens https://huggingface.co/settings/tokens



Once you add tokes into Colab, make sure to give Notebook access to the tokens

Fine-tuning results

```
...  Booting up the Base Intern (Untrained)...
Loading weights: 100%  100/100 [00:00<00:00, 438.43it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weight]
DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                     | Status     |
------------------------+------------+-
vocab_transform.weight  | UNEXPECTED |
vocab_layer_norm.bias   | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |
vocab_transform.bias    | UNEXPECTED |
vocab_projector.bias    | UNEXPECTED |
classifier.weight       | MISSING    |
classifier.bias         | MISSING    |
pre_classifier.bias     | MISSING    |
pre_classifier.weight   | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
 Booting up our Custom Expert (Fine-Tuned)...
Loading weights: 100%  104/104 [00:00<00:00, 504.32it/s, Materializing param=pre_classifier.weight]

============================================================
📝 TEST SENTENCE: 'I was so frustrated when my code crashed, but finding the bug made me so incredibly happy!'
============================================================

❌ BEFORE FINE-TUNING (Base Model):
Prediction: LABEL_0 (It has no idea what this means)
Confidence: 55.8% (Guessing blindly)

✅ AFTER FINE-TUNING (Our Custom Model):
Prediction: Joy 😄
Confidence: 99.3% (Highly confident!)
============================================================
```